Kỹ thuật Hai con trỏ & Cửa sổ trượt

Two Pointers & Sliding Window in Competitive Programming

Chuyên đề Thuật toán

July 14, 2026

Nội dung bài học

1. Mở đầu: Bài toán Two Sum
2. Định nghĩa và Phân loại
3. Kỹ thuật Cửa sổ trượt (Sliding Window)
4. Kinh nghiệm & Tổng kết

Bài toán mở đầu

Bài toán: Cặp số có tổng bằng X (Two Sum)

Cho mảng A gồm N phần tử **đã được sắp xếp tăng dần** và một số nguyên X.
Yêu cầu: Kiểm tra xem có tồn tại hai chỉ số i, j ($i \neq j$) sao cho $A[i] + A[j] = X$ hay không?

Cách tiếp cận ngây thơ (Brute Force):

- Sử dụng hai vòng lặp lồng nhau duyệt mọi cặp (i, j).
- Nếu $A[i] + A[j] = X$, in ra `YES`.

Sai lầm thường gặp

Độ phức tạp thuật toán là $\mathcal{O}(N^2)$. Nếu $N = 10^5$, số phép toán lên tới 10^{10}, chắc chắn bị Time Limit Exceeded (TLE) trong thi đấu!

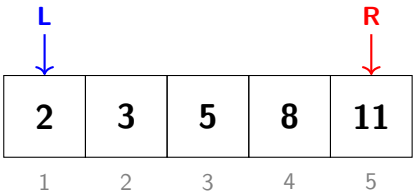
Ý tưởng

Vì mảng đã được sắp xếp ($A[1] \leq A[2] \leq \cdots \leq A[N]$), ta có thể sử dụng hai "con trỏ" (pointers) từ hai đầu mảng tiến lại gần nhau.

Quy tắc di chuyển:

- Khởi tạo $L = 1$ (đầu mảng) và $R = N$ (cuối mảng).
- Tính $S = A[L] + A[R]$:
  1. Nếu $S = X \Rightarrow$ Tìm thấy, kết thúc thuật toán.
  2. Nếu $S < X \Rightarrow$ Cần tăng tổng lên $\rightarrow$ Tăng L lên 1 ($L \leftarrow L + 1$).
  3. Nếu $S > X \Rightarrow$ Cần giảm tổng xuống $\rightarrow$ Giảm R đi 1 ($R \leftarrow R - 1$).
- Dừng lại khi $L \geq R$.

Minh họa hoạt động



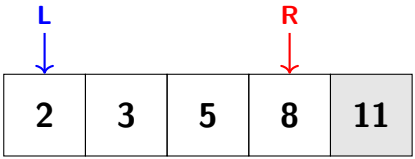
Quá trình chạy ($X = 11$)

Bước 1:

$L = 1, R = 5$

$A[1] + A[5] = 2 + 11 = 13 > 11$

$\rightarrow$ **Giảm** R.



Quá trình chạy

Bước 2:

$L = 1, R = 4$

$A[1] + A[4] = 2 + 8 = 10 < 11$

$\rightarrow$ **Tăng** L.

Cài đặt Two Sum bằng C++

```cpp
#include <bits/stdc++.h>
using namespace std;

bool hasTwoSum(vector<int>& a, int X) {
    int n = a.size();
    int L = 0, R = n - 1; // Khoi tao 2 con tro o hai dau

    while (L < R) { // Khong xet L = R vi phai phan biet
        int sum = a[L] + a[R];
        if (sum == X) return true; // Tim thay

        if (sum < X) {
            L++; // Tang L de tang tong
        } else {
            R--; // Giam R de giam tong
        }
    }
    return false; // Khong tim thay cap tong thoa man
}
```

Kỹ thuật Hai con trỏ (Two Pointers)

Khái niệm

Là kỹ thuật sử dụng hai biến chỉ số (pointers) để duyệt qua các cấu trúc dữ liệu tuyến tính (như mảng, chuỗi) một cách đồng thời, giúp tối ưu thời gian từ $\mathcal{O}(N^2)$ xuống $\mathcal{O}(N)$.

Con trỏ ngược chiều

- Một ở đầu, một ở cuối.
- Tiến lại gần nhau.
- **Bài toán:** Two Sum, Palindrome, Reverse Array.

Con trỏ cùng chiều

- Cả hai xuất phát từ đầu.
- Cùng di chuyển về một hướng.
- **Bài toán:** Gộp 2 mảng, Xóa phần tử trùng, **Sliding Window**.

Kỹ thuật Cửa sổ trượt (Sliding Window)

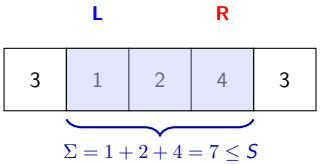
Bản chất

Sliding Window là một trường hợp đặc biệt của hai con trỏ **cùng chiều**. Ta dùng hai con trỏ L (trái) và R (phải) để duy trì một "cửa sổ" là mảng con liên tiếp $[L, R]$ thỏa mãn một tính chất nào đó.

Bài toán kinh điển

Cho mảng A gồm N số **nguyên dương** và một số S.
Hãy tìm **chiều dài lớn nhất** của một mảng con liên tiếp có tổng không vượt quá S.
Ví dụ: $A = [3, 1, 2, 4, 3], S = 7$. Output $= 3$ (mảng con $[1, 2, 4]$).

Hoạt động của Cửa sổ trượt



Ý tưởng

- **Mở rộng:** Di chuyển R sang phải để thêm phần tử vào cửa sổ.
- **Thu hẹp:** Nếu tổng vượt quá S, di chuyển L sang phải cho đến khi tổng $\leq S$.
- **Cập nhật:** Ghi nhận chiều dài lớn nhất $(R - L + 1)$ khi cửa sổ hợp lệ.

Cài đặt Sliding Window bằng C++

```cpp
int longestSubarray(vector<int>& a, int S) {
    int n = a.size();
    int L = 0, current_sum = 0, ans = 0;

    // R la con tro mo rong cua so
    for (int R = 0; R < n; R++) {
        current_sum += a[R]; // Them a[R] vao cua so

        // Neu vi pham dieu kien, thu hep ben trai
        while (current_sum > S) {
            current_sum -= a[L];
            L++;
        }

        // Cua so [L, R] luc nay hop le
        ans = max(ans, R - L + 1);
    }
    return ans;
}
```

Những điểm cần lưu ý

Sai lầm thường gặp

- **Quên kiểm tra $L \leq R$:** Trong một số bài toán có phần tử âm, L có thể vượt quá R. Cần code vòng lặp cẩn thận: `while(L <= R && ...)`.
- **Tràn số (Integer Overflow):** Biến `current_sum` phải dùng `long long` nếu tổng các phần tử vượt giới hạn số nguyên 32-bit.
- **Vị trí cập nhật kết quả:** Tùy bài toán mà cập nhật `ans` ở bên trong hay bên ngoài vòng lặp `while`.

Ghi chú

Sliding Window cổ điển ở slide trước chỉ đúng khi các phần tử trong mảng là **không âm**. Vì khi đó, việc thêm 1 phần tử (tăng R) chắc chắn làm hàm tổng **tăng lên** (tính đơn điệu).

Độ phức tạp & Kinh nghiệm

Độ phức tạp

- **Thời gian:** $\mathcal{O}(N)$.
  Tại sao không phải $\mathcal{O}(N^2)$ dù có vòng lặp lồng nhau?
  Vì cả con trỏ L và R đều chỉ di chuyển tối đa N bước theo **một chiều**. Tổng số bước lặp $\leq 2N$.
- **Không gian (Bộ nhớ):** $\mathcal{O}(1)$.
  Chỉ lưu vài biến trạng thái.

Kinh nghiệm thi đấu

Dấu hiệu nhận biết

- Đề bài yêu cầu tìm **dãy con liên tiếp** (subarray, substring).
- Tìm độ dài dài nhất/ngắn nhất.
- **Tính đơn điệu:** Khi cửa sổ rộng ra, hàm mục tiêu tăng (hoặc giảm). Nếu hẹp lại, hàm đổi chiều.

Tổng kết bài học

Kết luận

- **Two Pointers** và **Sliding Window** là vũ khí lợi hại để giảm độ phức tạp từ $\mathcal{O}(N^2)$ xuống $\mathcal{O}(N)$.
- Luôn tận dụng được khi dữ liệu đã được **sắp xếp** hoặc có **tính đơn điệu**.
- Mã nguồn thường rất ngắn, gọn, nhưng dễ sai sót ở các trường hợp biên (edge cases).